

Dunaújvárosi Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

Az ötlet rövid leírása:	1
Az ötlet lényege:	1
Hozzávalók és költségvetés.....	1
Működési elv	2
Kapcsolási rajz	2
Arduino IDE Kód.....	3
Fejlesztési lehetőségek	5
1. Wi-Fi és Felhő kapcsolat (Valódi IoT)	5
2. Automatizálás (Beavatkozás)	5
Önreflexió	6

Tantárgy neve: IoT
Projekt tervező: Morva Olivér
Projekt címe: Okos hőmérő
Osztály: 13.B
Dátum: 2025.12.10.

Az ötlet rövid leírása:

Az ötlet lényege:

Egy olcsó, Wi-Fi képes mikrokontroller méri a környezeti levegőt, majd az adatokat az interneten keresztül egy felhőplatformra küldi.

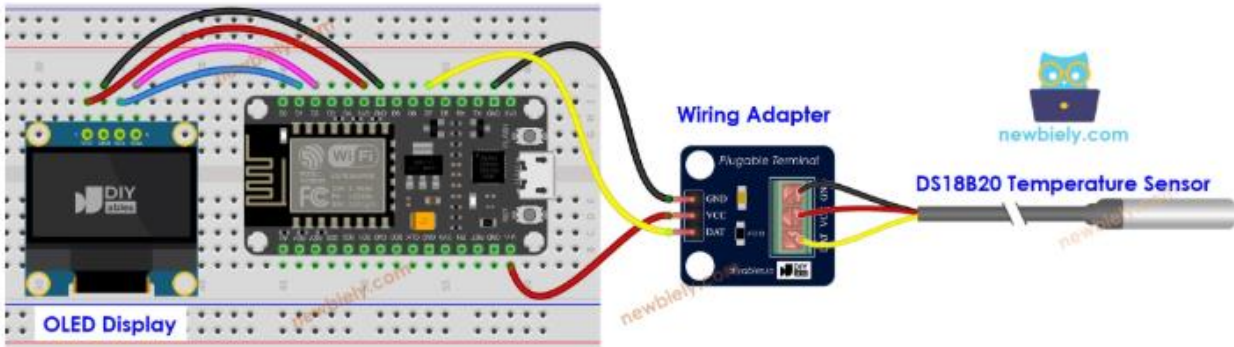
Hozzávalók és költségvetés

Alkatrész	Típus / Leírás	Ár (kb.)
Vezérlő egység	NodeMCU V3 (ESP8266) - Wi-Fi-vel	1 600 – 2 500 Ft
Szenzor	DHT22 (AM2302) - Pontosabb, mint a kék DHT11	2 000 – 3 500 Ft
Kábelek	10 db-os Jumper kábel készlet (Anyá-Anyá)	300 – 500 Ft
Próbapanel	Mini Breadboard (ha nem akarsz forrasztani)	500 – 900 Ft
Összesen		kb. 4 400 – 7 400 Ft

Működési elv

Az okos hőmérő működési elve három fő rétegre osztható: a fizikai mérésre, az adatfeldolgozásra és az internetes kommunikációra. A rendszer lényege az **adatátalakítás**: a szobában lévő fizikai változásból (melegsik a levegő) elektromos jel, abból digitális adat, majd végül a telefonodon látható grafikon lesz.

Kapcsolási rajz



Arduino IDE Kód

```
/*  
 * This ESP8266 NodeMCU code was developed by newbiely.com  
 *  
 * This ESP8266 NodeMCU code is made available for public use without any restriction  
 *  
 * For comprehensive instructions and wiring diagrams, please visit:  
 * https://newbiely.com/tutorials/esp8266/esp8266-temperature-sensor-oled  
 */  
  
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SSD1306.h>  
#include <OneWire.h>  
#include <DallasTemperature.h>  
  
#define OLED_WIDTH 128 // OLED display width, in pixels  
#define OLED_HEIGHT 64 // OLED display height, in pixels  
  
#define SENSOR_PIN 2 // The ESP8266 pin connected to DS18B20 sensor's DQ pin  
  
Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, -1); // create SSD1306  
display object connected to I2C  
  
OneWire oneWire(SENSOR_PIN); // setup a oneWire instance  
DallasTemperature DS18B20(&oneWire); // pass oneWire to DallasTemperature library  
  
String temperature_str;  
  
void setup() {
```

```
Serial.begin(9600);
```

```
// initialize OLED display with address 0x3C for 128x64
```

```
if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
```

```
    Serial.println(F("SSD1306 allocation failed"));
```

```
    while (true);
```

```
}
```

```
delay(2000);    // wait for initializing
```

```
oled.clearDisplay(); // clear display
```

```
oled.setTextSize(2);    // text size
```

```
oled.setTextColor(WHITE); // text color
```

```
oled.setCursor(0, 10);   // position to display
```

```
DS18B20.begin(); // initialize the sensor
```

```
temperature_str.reserve(10); // to avoid fragmenting memory when using String
```

```
}
```

```
void loop() {
```

```
    DS18B20.requestTemperatures();    // send the command to get temperatures
```

```
    float temperature_C = DS18B20.getTempCByIndex(0); // read temperature in Celsius
```

```
    temperature_str = String(temperature_C, 2); // two decimal places
```

```
    temperature_str += char(247) + String("C");
```

```
    Serial.println(temperature_str); // print the temperature in Celsius to Serial Monitor
```

```
    oled_display_center(temperature_str);
```

```
}
```

```
void oled_display_center(String text) {  
    int16_t x1;  
    int16_t y1;  
    uint16_t width;  
    uint16_t height;  
  
    oled.getTextBounds(text, 0, 0, &x1, &y1, &width, &height);  
  
    // center the display both horizontally and vertically  
    oled.clearDisplay(); // clear display  
    oled.setCursor((OLED_WIDTH - width) / 2, (OLED_HEIGHT - height) / 2);  
    oled.println(text); // text to display  
    oled.display();  
}
```

Fejlesztési lehetőségek

1. Wi-Fi és Felhő kapcsolat (Valódi IoT)

A jelenlegi kódod csak helyben (az OLED kijelzőn) mutatja az adatokat. A legfontosabb lépés, hogy az adatokat **felhőbe** küldd.

- **Blynk vagy ThingSpeak:** Ezekkel az ingyenes platformokkal percek alatt készíthetsz egy mobilalkalmazást, ahol grafikonon látod a hőmérséklet alakulását az elmúlt 24 órában.
- **ESP8266/ESP32 csere:** Az Arduino Uno helyett használj Wi-Fi képes vezérlőt, így bárhonnan ránézhetsz a szobádra.

2. Automatizálás (Beavatkozás)

Ne csak mérj, cselekedj is! Adj a rendszerhez egy **Relé modult**:

- **Párátlanítás:** Ha a páratartalom 70% fölé megy, a rendszer automatikusan kapcsoljon be egy ventilátort vagy párátlanítót.
- **Fűtésvezérlés:** Ha 18°C alá esik a hő, kapcsoljon be egy kiegészítő fűtőtestet.

Önreflexió

Az elmúlt évek során az egyik legmeghatározóbb élményem ez a tantárgy volt. Bár kezdetben kihívást jelentett a hardveres eszközök és a programkód összehangolása, a gyakorlati feladatok során gyorsan ráéreztem az **IoT (Dolgok Internete)** világának logikájára.

Különösen élveztem a projektalapú megközelítést. Az **okos hőmérő és páratartalom-mérő** állomás megépítése során nemcsak elméleti tudást szereztem, hanem valódi sikerélményt is, amikor az OLED kijelzőn először megjelentek a szenzor adatai, vagy amikor a Buzzer és az RGB LED megfelelően reagált a környezeti változásokra. Megértettem, hogyan válik a fizikai világ jeleiből digitális adat, és ez az "alkotás" folyamata az, ami leginkább motivált.

A jövőt tekintve biztos vagyok benne, hogy továbbra is foglalkozni szeretnék ezzel a területtel. Látom a lehetőséget a projekt továbbfejlesztésében – legyen szó a **felhőalapú adattárolásról** vagy az **otthonautomatizálási rendszerekbe** való integrálásról. Ez a tárgy rávezetett arra, hogy a programozás sokkal izgalmasabb, amikor fizikai eszközöket tudunk vele vezérelni és okosabbá tenni a környezetünket.

Összességében egy inspiráló kurzust zárhattam le, amely megalapozta a szakmai érdeklődésemet, és kíváncsian várom, milyen komplexebb IoT megoldásokat tudok majd a jövőben megvalósítani.